

Reviewer Pack — Minimal Rank of Tropical Curves Bounds AC^0 Parity Depth

Entry 97faaad75eed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 12:48:09 UTC

Minimal Rank of Tropical Curves Bounds AC^0 Parity Depth

Entry ID: 97faaad75eed **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 12:48:09 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): AC^0 Circuit Complexity

Statement:

{‘quantitative’: ‘For any AC^0 circuit C computing PARITY on n inputs, the minimal rank of the corresponding tropical curve defined over the tropical semiring is at least $c \cdot \log(n)$ ’, ‘invariant’: ‘ $\psi(C) := \min \text{rank}(\text{TropicalCurve}(C))$ ’, ‘correlation’: ‘where $\psi(C) > c \cdot \log(\text{size}(C))$ for any C computing PARITY’}

Rationale (proposer’s reasoning):

{‘exposure’: ‘Tropical geometry provides a natural setting to study the geometric properties of computational problems, and its connections to circuit complexity are relatively unexplored.’, ‘structure’: ‘The minimal rank of tropical curves could capture intrinsic structural information about circuits that is not captured by traditional size or depth measures.’, ‘application’: ‘Understanding the connection between tropical geometry and AC^0 parity depth may reveal new insights into the nature of computation and its bounds.’}

Taxonomy category: AC^0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 32298fe48bd13a57

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of tropical curves bounds AC^0 parity depth: The conjecture is supported if, for at least 80% of randomly generated n -input AC^0 circuits, the minimal rank $\psi(C)$ of the corresponding tropical curve is greater than or equal to $c \cdot \log(n)$, where c is a constant and $\log(n)$ is calculated based on the size of the circuit.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        circuit = []
        for _ in range(n):
            gate_type = random.choice(['AND', 'OR'])
            inputs = [random.choice('x' + ''.join(map(str, range(1, n+1)))) for _ in range(random.randint(1, n))]
            circuit.append((gate_type, inputs))
        return circuit

    def evaluate_circuit(circuit, input_values):
        stack = []
        for gate_type, inputs in reversed(circuit):
            if gate_type == 'AND':
                result = True
                for inp in inputs:
                    if inp[0] == 'x':
                        result &= input_values[ord(inp[1:]) - ord('1')]
                else:
                    result &= bool(int(inp))
                stack.append(result)
            elif gate_type == 'OR':
                result = False
                for inp in inputs:
                    if inp[0] == 'x':
                        result |= input_values[ord(inp[1:]) - ord('1')]
                else:
                    result |= bool(int(inp))
                stack.append(result)
        return stack.pop()

    def tropical_rank(circuit):
```

```

n = len(circuit)
matrix = [[-math.inf for _ in range(n)] for _ in range(n)]
for i, (gate_type, inputs) in enumerate(circuit):
    if gate_type == 'AND':
        for inp in inputs:
            if inp[0] == 'x':
                j = ord(inp[1:]) - ord('1')
                matrix[i][j] = 0
                matrix[j][i] = 0
    elif gate_type == 'OR':
        for inp in inputs:
            if inp[0] == 'x':
                j = ord(inp[1:]) - ord('1')
                matrix[i][j] = 1
                matrix[j][i] = 1

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if A[j][i] > A[max_row][i]:
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(n):
            A[i][j] /= A[i][i]
        for k in range(m):
            if k != i:
                factor = A[k][i]
                for j in range(n):
                    A[k][j] -= factor * A[i][j]
    return A

gaussian_elimination(matrix)

rank = 0
for row in matrix:
    if any(x > -math.inf for x in row):
        rank += 1
return rank

def generate_input_values(n):
    input_values = {}
    for i in range(1, n+1):
        input_values[f'x{i}'] = random.choice([0, 1])
    return input_values

def compute_metric_value(circuit, input_values):
    result = evaluate_circuit(circuit, input_values)
    rank = tropical_rank(circuit)
    return rank

n_values = [5, 10, 15, 20, 30, 40]
metric_values = []
for n in n_values:
    circuit = generate_circuit(n)

```

```

        input_values = generate_input_values(n)
        metric_value = compute_metric_value(circuit, input_values)
        metric_values.append(metric_value)

    mean_metric_value = sum(metric_values) / len(metric_values)
    std_metric_value = (sum((x - mean_metric_value) ** 2 for x in metric_values) / len(metric_values)) ** 0.5

    conjecture_holds = all(value >= math.log(n, 2) for n, value in zip(n_values, metric_values))

    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "tropical_rank",
        "metric_value": mean_metric_value,
        "instances_tested": len(metric_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if none provided

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = (sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results)) ** 0.5
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47e80a40.py", line 132, in <module>
    trial_result = run_trial(seed)
    ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47e80a40.py", line 108, in run_trial
    metric_value = compute_metric_value(circuit, input_values)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47e80a40.py", line 99, in compute_metri
    result = evaluate_circuit(circuit, input_values)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47e80a40.py", line 36, in evaluate_circ
    result &= input_values[ord(inp[1:]) - ord('1')]
    ^^^^^^^^^^^^^

```

TypeError: ord() expected a character, but string of length 0 found

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the conjecture’s support condition of at least 80% of randomly generated n-input AC⁰ circuits. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11122
2	preregistration	ollama_remote	glm4:latest	0	0	6013
3	novelty	ollama_remote	glm4:latest	0	0	4759
4	novelty	ollama_remote	glm4:latest	0	0	5112
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11082
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12233
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13272
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14452
9	judge	ollama_remote	glm4:latest	0	0	9172

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 87218 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/97faaad75eed.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/97faaad75eed.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/97faad75eed.tar.gz (if generated)